# A Sample Lecture on Graph Search

## From Textbook Pseudocode to Reproducible Experiments

Your Name

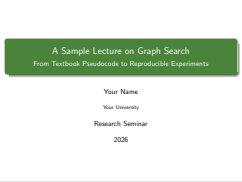
Your University

Research Seminar

2026








Hi everyone. **Thanks for being here.** Today I am walking through a sample lecture on graph search.

We will explore where the classic algorithms come from and what they mean for real workloads.

The big idea is: if we want results people can trust, we need structure, not just speed.

Before diving in, let me quickly outline what we will cover today.

# Content

1 Introduction
2 Related Work
3 Approach
4 Foundations
5 Evaluation
6 Conclusion





Content
Introduction
Related Work
Approach
Foundations
Evaluation
Conclusion

Six parts. Spend most of the time on the approach and the evaluation. Skip the foundations if the audience already knows BFS.

# Why Graph Search Still Matters

- Road networks, citation graphs and dependency trees





Why Graph Search Still Matters
- Road networks, citation graphs and dependency trees

Reveal one bullet at a time. Ask who has tried to reproduce a benchmark.

# Why Graph Search Still Matters

- Road networks, citation graphs and dependency trees
- Small changes to the frontier change the whole cost





Why Graph Search Still Matters
- Road networks, citation graphs and dependency trees
- Small changes to the frontier change the whole cost

Reveal one bullet at a time. Ask who has tried to reproduce a benchmark.

# Why Graph Search Still Matters

- Road networks, citation graphs and dependency trees
- Small changes to the frontier change the whole cost
- Reproducing a published result is harder than it looks





Why Graph Search Still Matters
- Road networks, citation graphs and dependency trees
- Small changes to the frontier change the whole cost
- Reproducing a published result is harder than it looks

Reveal one bullet at a time. Ask who has tried to reproduce a benchmark.

# Research Questions

**RQ1**

Which frontier gives the best trade-off on sparse graphs?








Both questions are answered in the evaluation section. Point forward to the summary table.

# Research Questions

**RQ1**

Which frontier gives the best trade-off on sparse graphs?

**RQ2**

How stable are the results across hardware?





Research Questions
RQ1
Which frontier gives the best trade-off on sparse graphs?
RQ2
How stable are the results across hardware?

Both questions are answered in the evaluation section. Point forward to the summary table.

# A Short History

- Shortest paths with a priority queue [1]
- Heuristic search with admissible estimates [2]
- Community benchmarks for reproducibility [3]





A Short History
- Shortest paths with a priority queue [1]
- Heuristic search with admissible estimates [2]
- Community benchmarks for reproducibility [3]

The citations are links. In TeXora, hover one to peek at the reference without leaving this slide.

# The Core Recurrence

$$d(v) \;=\; \min_{(u,v)\in E} \big(d(u) + w(u,v)\big)$$





The Core Recurrence
$d(v) = \min_{(u,v)\in E} (d(u) + w(u,v))$

First the plain recurrence, then add the heuristic.
Stress that the estimate must never overshoot.

# The Core Recurrence

$$d(v) \;=\; \min_{(u,v)\in E} \big(d(u) + w(u,v)\big)$$

$$f(v) \;=\; d(v) + h(v), \qquad h(v) \le d^*(v)$$





The Core Recurrence
$d(v) = \min_{(u,v)\in E}(d(u) + w(u,v))$
$f(v) = d(v) + h(v), \quad h(v) \le d^*(v)$

First the plain recurrence, then add the heuristic.
Stress that the estimate must never overshoot.

# Search Pipeline








Search Pipeline
Graph

Build the diagram step by step. The loop back is the part people forget.

# Search Pipeline





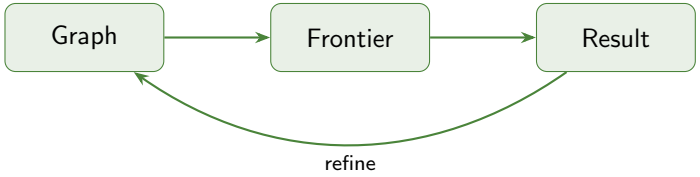



Search Pipeline




Build the diagram step by step. The loop back is the part people forget.

# Search Pipeline











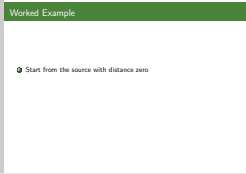
Build the diagram step by step. The loop back is the part people forget.

# Worked Example

1. Start from the source with distance zero







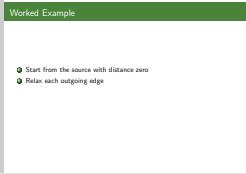

Use the pen to trace the order on the board. Use the laser to point at step three.

# Worked Example

1. Start from the source with distance zero
2. Relax each outgoing edge








Use the pen to trace the order on the board. Use the laser to point at step three.

# Worked Example

1. Start from the source with distance zero
2. Relax each outgoing edge
3. Pop the smallest tentative distance





Worked Example
1. Start from the source with distance zero
2. Relax each outgoing edge
3. Pop the smallest tentative distance

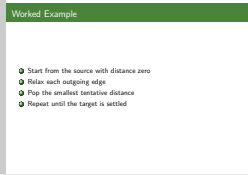
Use the pen to trace the order on the board. Use the laser to point at step three.

# Worked Example

1. Start from the source with distance zero
2. Relax each outgoing edge
3. Pop the smallest tentative distance
4. Repeat until the target is settled








Use the pen to trace the order on the board. Use the laser to point at step three.

# Complexity at a Glance

| Method | Time | Space |
|---|---|---|
| Breadth-first | $O(V + E)$ | $O(V)$ |
| Binary heap | $O(E \log V)$ | $O(V)$ |
| Fibonacci heap | $O(E + V \log V)$ | $O(V)$ |





Complexity at a Glance
Method Time Space
Breadth-first $O(V + E)$ $O(V)$
Binary heap $O(E \log V)$ $O(V)$
Fibonacci heap $O(E + V \log V)$ $O(V)$

Mention that the theoretical winner is rarely the practical winner.

# Experimental Setup

- Twelve sparse graphs, three dense graphs
- Five repetitions, median reported
- Fixed seeds and pinned library versions



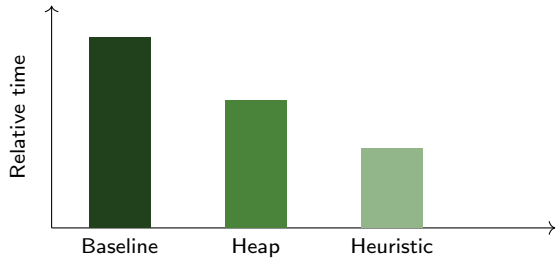


Experimental Setup
- Twelve sparse graphs, three dense graphs
- Five repetitions, median reported
- Fixed seeds and pinned library versions

Keep this short. Details are in the backup slide if someone asks.

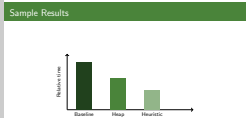
Backup: full setup.

# Sample Results








These numbers are invented placeholders for the demo. The takeaway: the heuristic halves the runtime.

# Take-Home Messages

1. Choose the frontier for your graph, not for the textbook
2. Publish seeds, versions and raw data
3. Questions welcome





Take-Home Messages
Choose the frontier for your graph, not for the textbook
Publish seeds, versions and raw data
Questions welcome

Pause here. Leave this slide up during the questions.

# Backup: Full Setup

- Sixteen cores, 64 GB of memory
- Compiler optimisation level three
- Timings exclude graph loading





Backup: Full Setup
- Sixteen cores, 64 GB of memory
- Compiler optimisation level three
- Timings exclude graph loading

Only show this if someone asks about the hardware.

# References


[1] E. W. Dijkstra. A note on two problems in connexion with graphs. *Numerische Mathematik*, 1959.

[2] P. E. Hart, N. J. Nilsson and B. Raphael. A formal basis for the heuristic determination of minimum cost paths. *IEEE Trans. SSC*, 1968.

[3] Placeholder Authors. Sample benchmark collection. *Example Proceedings*, 2020.






References

[1] E. W. Dijkstra. A note on two problems in connexion with graphs. Numerische Mathematik, 1959.

[2] P. E. Hart, N. J. Nilsson and B. Raphael. A formal basis for the heuristic determination of minimum cost paths. IEEE Trans. SSC, 1968.

[3] Placeholder Authors. Sample benchmark collection. Example Proceedings, 2020.

Reference slide. Reached from the citation links.